

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (*BL5*)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:40

Annexure A

Industry Problem Solving Task

Code of Conduct:

I D Varshini (192320007)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Industry Problem Solving Task

1. Smart Banking and Financial Management System

A financial institution develops a **Smart Banking and Financial Management System** to manage customer accounts, fund transfers, loan processing, bill payments, and transaction history. The application follows a layered object-oriented architecture consisting of a presentation layer, service layer, domain layer, and data access layer connected to an OODBMS.

The system uses the **Factory Pattern** to create different account types, the **Strategy Pattern** for loan-interest and payment calculation policies, and the **Observer Pattern** for transaction and fraud alerts. A **Repository Pattern** is used to manage communication with persistent banking objects. The architecture is designed to support future services such as investment management and digital wallets.

Question:

Analyze how object-oriented principles, layered architecture, and design patterns contribute to the scalability, security, flexibility, and maintainability of the banking system. Explain how the application layers communicate with the OODBMS to store and retrieve persistent banking objects.

Answer:

System Analysis

The proposed banking system is strong because responsibilities are divided between domain objects, services, repositories and reusable design-pattern components. The object model represents real banking concepts such as Customer, Account, Transaction and Loan, while the service layer coordinates complete use cases such as fund transfer, loan processing and bill payment. This prevents a few large classes from becoming responsible for every operation.

Object-Oriented Principles

- Encapsulation protects sensitive state such as account balance, loan status and transaction data. State is changed only through controlled operations such as `credit()`, `debit()`, `transfer()` or `approveLoan()`, where validation and authorization rules can be enforced.

- Abstraction separates what a component does from how it does it. Service and repository interfaces hide persistence details, while strategy interfaces hide loan-interest and payment calculations.
- Inheritance and polymorphism allow SavingsAccount, CurrentAccount and future account types to share the Account contract while implementing type-specific rules. SOLID-style separation keeps security, business rules and persistence independently changeable.

Layered Architecture

- Presentation layer: authenticates the user, collects input, performs basic format validation and invokes application services. It never executes OODBMS queries directly.
- Service layer: implements use cases such as openAccount(), transferFunds(), processLoan() and payBill(). It performs authorization, business validation and transaction coordination.
- Domain layer: contains Customer, Account, Transaction, Loan, Payment and their business invariants. This layer represents the core banking model.
- Data access layer: repository implementations isolate OODBMS sessions, queries, object identifiers and transaction commits from the rest of the application.

Application of Design Patterns

- Factory Pattern: AccountFactory creates SavingsAccount, CurrentAccount or another Account subtype from a request. New account types can be added without scattering constructor logic across the application.
- Strategy Pattern: LoanInterestStrategy and PaymentCalculationStrategy encapsulate variable algorithms. A new interest policy, repayment rule or payment fee policy can be introduced as another strategy instead of rewriting service classes.
- Observer Pattern: transaction and fraud events notify observers such as SMSAlert, EmailAlert and FraudMonitor. The transaction service publishes an event without being tightly coupled to every notification channel.
- Repository Pattern: AccountRepository, TransactionRepository and LoanRepository expose domain-oriented operations such as findById(), save() and findTransactions(), keeping OODBMS code behind a stable interface.

Important Classes and Responsibilities

Class / Component	Key Attributes	Key Responsibilities / Methods
Customer	customerId, name, contact	viewAccounts(), requestLoan()
Account	accountNo, balance, status	credit(), debit(), getBalance()
Transaction	transactionId, amount, type, timestamp	validate(), post()
Loan	loanId, principal, rate, status	calculateInterest(), approve()
BankingService	repositories, strategies	transferFunds(), processLoan(), payBill()
Repository interfaces	persistent-object access	findById(), save(), update()

OODBMS Interaction and Persistence Flow

A user action flows Presentation -> Service -> Domain -> Repository -> OODBMS. For example, a transfer request reaches TransferService, which loads the source and destination Account objects through AccountRepository. The service validates ownership, balance and limits, asks the domain objects to debit/credit, creates a Transaction object, and passes the modified object graph back to the repositories.

The repository opens an OODBMS transaction, stores the changed Account and Transaction objects, preserves their object identities and relationships, and commits atomically. Retrieval performs the reverse path: the repository queries by object identity or domain field, returns domain objects to the service, and the presentation layer receives only the required view data.

Evaluation of Quality Attributes

- Scalability: new services such as digital wallets or investments can be introduced through new services, domain classes and repositories without redesigning existing modules.
- Security: centralized service-layer authorization, encapsulated domain state and a controlled persistence boundary reduce unauthorized direct access to banking objects.
- Flexibility: factories, strategies and observers make account types, policies and alert channels replaceable at runtime or through configuration.

- Maintainability/testability: layers can be tested independently; repository and strategy interfaces can be replaced by mocks, and changes to the OODBMS do not leak into UI or domain code.

UML / Layered Architecture Diagram

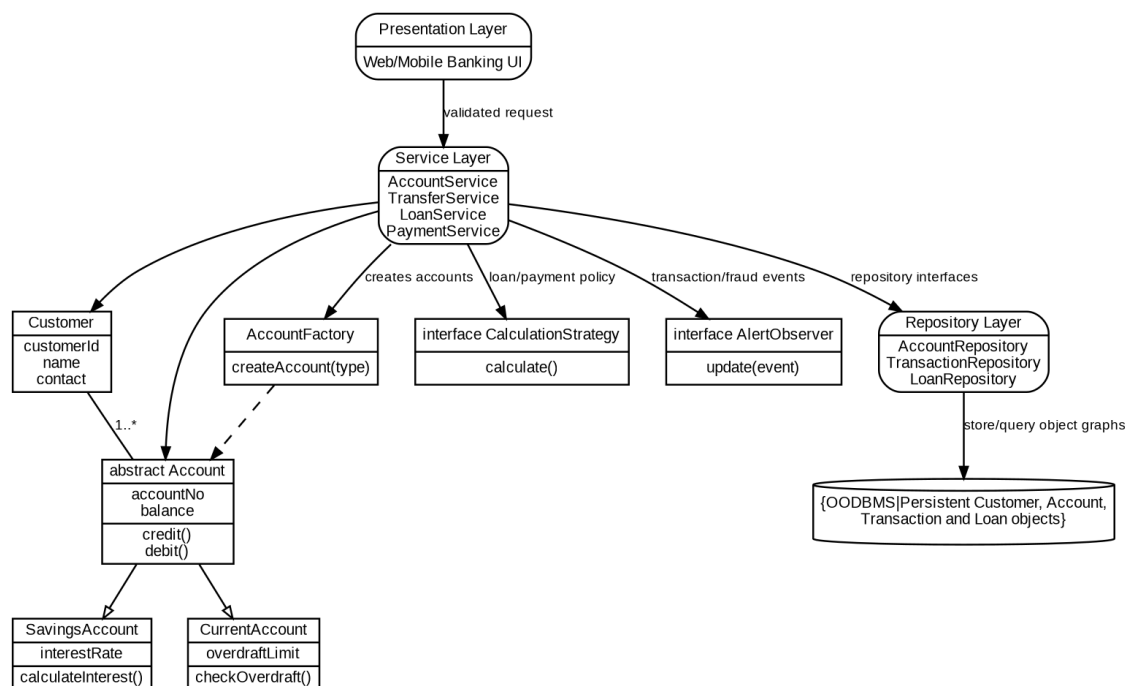


Figure 1: Object-oriented layered design for the system.

Conclusion

Thus, the architecture converts the banking application into a set of cohesive, loosely coupled components. Persistent banking objects are accessed only through repositories, while patterns isolate the main areas that are expected to vary, giving the system a safe path for future expansion.

2. Smart Retail and Inventory Management System

A retail organization develops a **Smart Retail and Inventory Management System** to manage products, suppliers, customers, purchase orders, sales transactions, and warehouse stock. The system follows a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system applies the **MVC Pattern** for managing user interfaces, the **Factory Pattern** for creating different product categories, and the **Observer Pattern** to notify managers when

inventory reaches a minimum threshold. The **DAO Pattern** handles communication with persistent inventory objects. The architecture allows future integration of automated stock prediction and robotic warehouse management.

Question:

Evaluate how object-oriented design principles, layered architecture, and design patterns improve the flexibility, scalability, and maintainability of the retail system. Explain how the different layers interact with the OODBMS for efficient inventory persistence and retrieval.

Answer:

System Analysis

The retail system benefits from an object-oriented model because products, suppliers, customers, purchase orders, sales and warehouse stock have different responsibilities but must collaborate. Layering prevents user-interface logic, stock rules and OODBMS code from being mixed, while the selected patterns isolate the parts of the system that change most often.

Object-Oriented Principles

- Encapsulation keeps stock quantity, reorder level and product pricing inside domain objects and allows updates only through controlled methods such as `adjustStock()`, `receiveStock()` and `recordSale()`.
- Abstraction is provided by service, DAO and observer interfaces. Controllers depend on business services rather than database code, and business services depend on DAO contracts rather than a concrete OODBMS API.
- Polymorphism lets product subtypes such as `GroceryProduct`, `ElectronicProduct` and `ApparelProduct` use common `Product` operations while applying category-specific behavior. This supports Open/Closed Principle and reduces repeated conditional logic.

Layered Architecture

- Presentation/MVC layer: Views display catalogue, stock and sales data; Controllers receive user actions and translate them into business-service calls.
- Business layer: `InventoryService`, `OrderService` and `SalesService` validate operations, coordinate stock changes and enforce purchasing/sales rules.
- Domain layer: `Product`, `Supplier`, `Customer`, `InventoryItem`, `PurchaseOrder` and `Sale` model business state and relationships.

- Data access layer: ProductDAO, InventoryDAO, OrderDAO and SaleDAO perform persistence operations and hide OODBMS query/session details.

Application of Design Patterns

- MVC Pattern separates View, Controller and Model responsibilities, so the interface can change without rewriting inventory rules or persistence logic.
- Factory Pattern centralizes creation of category-specific Product objects. Adding a new category requires a new subtype/factory branch rather than constructor logic in many screens and services.
- Observer Pattern connects InventoryItem threshold events to StockManagerNotifier, PurchaseReorderService or dashboard alerts. The stock object does not need to know every receiver.
- DAO Pattern gives each persistent object family a clear access component, allowing optimized queries and replacing the OODBMS implementation without changing controllers or domain rules.

Important Classes and Responsibilities

Class / Component	Key Attributes	Key Responsibilities / Methods
Product	productId, name, price, category	updatePrice(), getDetails()
InventoryItem	product, quantity, reorderLevel	adjustStock(), isBelowThreshold()
Supplier	supplierId, name, contact	supply(), updateDetails()
PurchaseOrder	orderId, supplier, items, status	addItem(), approve(), receive()
Sale	saleId, customer, items, total	calculateTotal(), complete()
InventoryDAO	OODBMS session	findByProduct(), save(), updateStock()

OODBMS Interaction and Persistence Flow

For a sale, the View sends the action to SalesController, which calls SalesService. The service obtains Product and InventoryItem objects through DAOs, verifies availability, creates the Sale, decreases stock through domain methods and triggers the low-stock observer if necessary.

The DAO persists the modified InventoryItem and Sale object graph in the OODBMS and commits the unit of work. When the dashboard requests stock, InventoryDAO queries persistent InventoryItem objects and their linked Product/Supplier objects, returns them to the service, and the controller prepares them for the View. Direct object persistence avoids manually reconstructing every relationship in presentation code.

Evaluation of Quality Attributes

- Flexibility: category creation, UI behavior and low-stock reactions are isolated in Factory, MVC and Observer components.
- Scalability: business services and DAOs can be extended for multiple warehouses, high transaction volumes, stock prediction or robotic warehouse modules.
- Maintainability: one layer owns each concern, so a UI redesign or OODBMS change does not require rewriting business rules.
- Efficiency: DAOs centralize frequently used inventory queries and can use object identity, lazy loading/caching and transaction boundaries appropriately.

UML / Layered Architecture Diagram

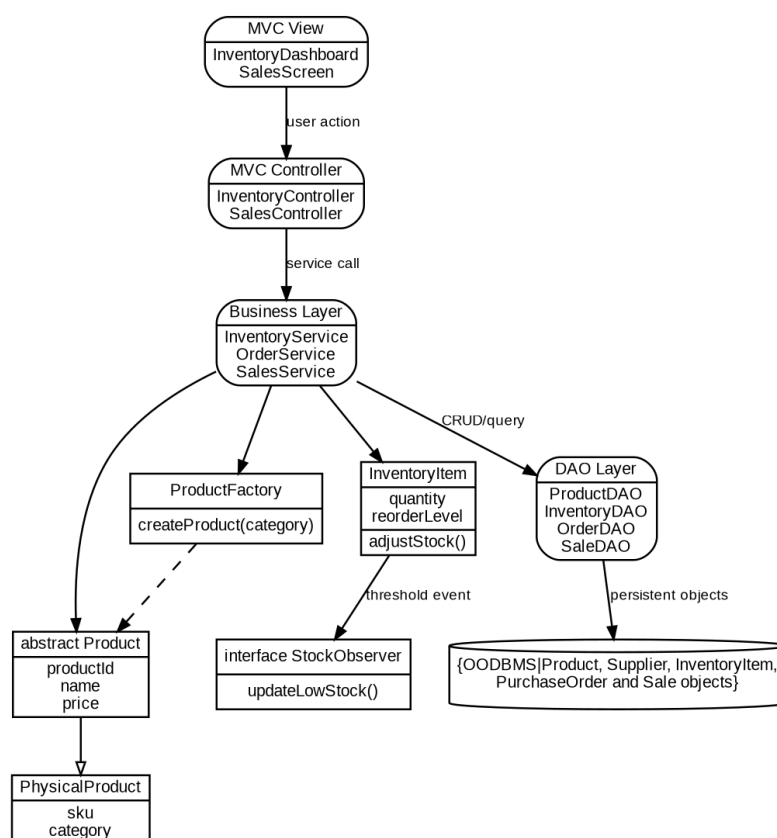


Figure 2: Object-oriented layered design for the system.

Conclusion

The redesign gives the retail system a stable core while keeping user interfaces, product categories, notifications and persistence replaceable. This is particularly suitable for future automated stock prediction because the new analytics service can consume the same domain and DAO interfaces.

3. Smart Hotel and Hospitality Management System

A hotel chain develops a **Smart Hotel Management System** to manage guest registration, room reservations, check-in/check-out, payments, room services, and customer feedback. The application uses a layered object-oriented architecture consisting of presentation, service, domain, and data access layers connected to an OODBMS.

The system implements the **Strategy Pattern** for different pricing and discount policies, the **Factory Pattern** for creating room and service objects, and the **Observer Pattern** for reservation and service notifications. A **Repository Pattern** is used to simplify communication between business components and persistent hotel objects. Future enhancements include smart-room automation and personalized guest services.

Question:

Analyze how object-oriented principles and design patterns support flexibility, reusability, and maintainability in the hotel management system. Explain how the layered architecture coordinates reservation processing and communicates with the OODBMS for persistent data management.

Answer:

System Analysis

In the hotel system, guest registration, room availability, reservations, pricing, payments and room services change at different rates. Object-oriented decomposition keeps these concepts independent, while a layered architecture provides a clear path from guest/reception screens to domain rules and finally to persistent hotel objects.

Object-Oriented Principles

- Encapsulation protects room status, reservation dates and payment state. A room cannot be marked occupied or a reservation confirmed by arbitrary UI code; these changes occur through domain/service operations.

- Abstraction through PricingStrategy and repository interfaces allows pricing rules and persistence technology to vary independently of ReservationService.
- Polymorphism enables room types and service types to share common interfaces while implementing their own rate, capacity or service behavior. Composition is used to associate Guest, Reservation, Room, Payment and optional room services.

Layered Architecture

- Presentation layer: BookingUI, GuestApp and ReceptionUI capture requests and display availability, reservation status and bills.
- Service layer: ReservationService coordinates search, booking, modification and cancellation; CheckInService and PaymentService handle their focused workflows.
- Domain layer: Guest, Room, Reservation, Payment, RoomService and Feedback contain hotel data and business rules.
- Data access layer: repositories provide domain-oriented operations and map persistent hotel object graphs to the OODBMS.

Application of Design Patterns

- Strategy Pattern encapsulates seasonal pricing, corporate discounts, loyalty discounts and other policies behind calculatePrice(). ReservationService selects the appropriate strategy without changing its workflow.
- Factory Pattern creates room and service objects using a common creation point. New room categories or hotel services can be added without duplicating construction logic.
- Observer Pattern sends reservation confirmations, check-in reminders and room-service notifications to registered observers such as guest messaging and staff dashboards.
- Repository Pattern hides object retrieval and storage, so ReservationService works with RoomRepository and ReservationRepository rather than OODBMS query code.

Important Classes and Responsibilities

Class / Component	Key Attributes	Key Responsibilities / Methods
Guest	guestId, name, contact, preferences	register(), updateProfile()
Room	roomNo, type, baseRate, status	isAvailable(), changeStatus()

Reservation	reservationId, dates, status	confirm(), cancel(), modify()
Payment	paymentId, amount, method, status	pay(), refund()
ReservationService	repositories, pricingStrategy	searchRooms(), createReservation(), cancelReservation()
ReservationRepository	OODBMS access	findById(), save(), findByGuest()

OODBMS Interaction and Persistence Flow

Reservation processing begins when the UI calls ReservationService with the dates and room preference. The service asks RoomRepository for available persistent Room objects, applies PricingStrategy, validates the selected room, creates a Reservation linked to the Guest and Room, and registers notification observers.

ReservationRepository stores the new Reservation object and its relationships in an OODBMS transaction. PaymentRepository later stores Payment linked to the reservation. On retrieval, the repository loads the reservation object graph needed for check-in, billing or history and returns domain objects to the service; no screen contains database commands.

Evaluation of Quality Attributes

- **Flexibility:** pricing/discount strategies and notification channels can change without altering reservation logic.
- **Reusability:** the same domain, service and repository components serve web booking, mobile apps and front-desk interfaces.
- **Maintainability:** responsibilities are localized, reducing regression risk and making unit tests straightforward.
- **Future readiness:** smart-room automation and personalized guest services can be added as observers/services that use existing reservation and guest abstractions.

UML / Layered Architecture Diagram

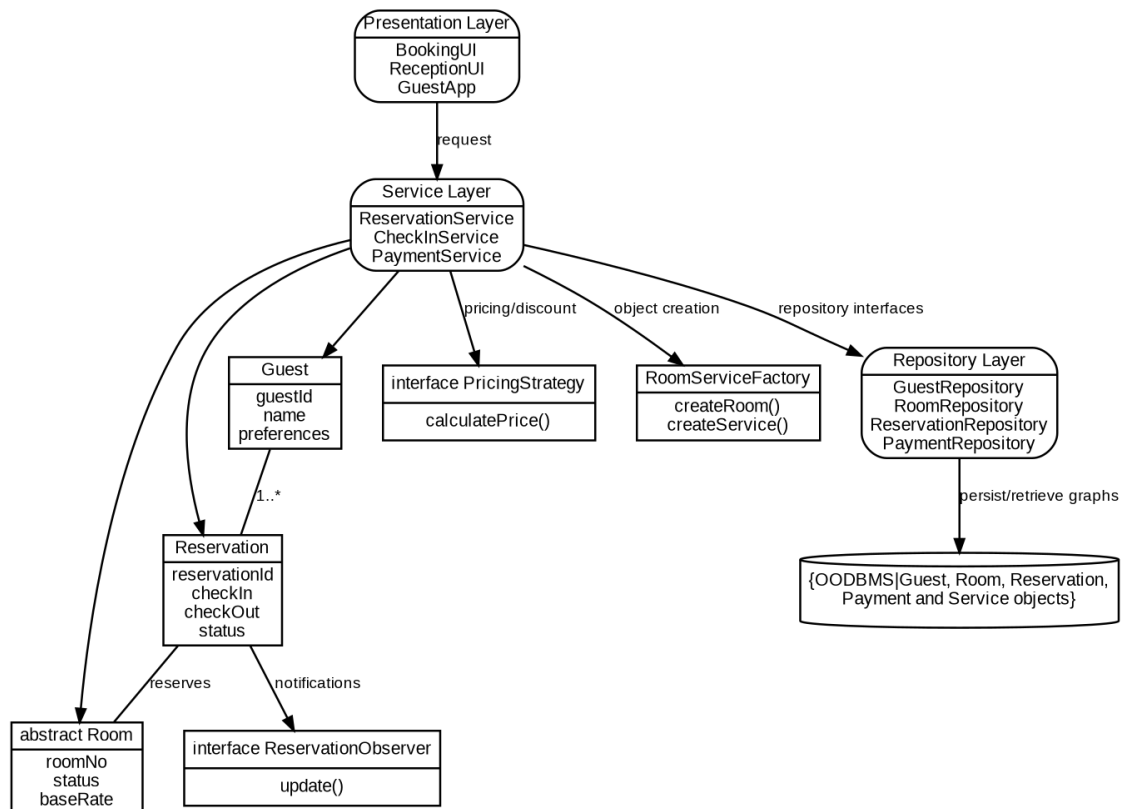


Figure 3: Object-oriented layered design for the system.

Conclusion

The layered design therefore keeps reservation coordination cohesive and makes persistent hotel objects accessible through a single repository boundary. The system can grow in features while retaining the same stable domain model and service contracts.

4. Smart Logistics and Delivery Management System

A logistics company develops a **Smart Logistics and Delivery Management System** to manage shipment registration, warehouse operations, vehicle allocation, delivery tracking, and customer notifications. The system adopts a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system uses the **Strategy Pattern** for selecting optimal delivery routes, the **Factory Pattern** for creating different shipment types, and the **Observer Pattern** for real-time delivery status notifications. A **DAO Pattern** manages persistent shipment and vehicle information. The architecture is designed to support future integration with autonomous delivery vehicles and AI-based route optimization.

Question:

Examine how object-oriented design principles, layered architecture, and design patterns improve the adaptability, scalability, and maintainability of the logistics system. Explain how the layers communicate with the OODBMS to manage persistent shipment, vehicle, and delivery information.

Answer:

System Analysis

The logistics system must coordinate shipments, warehouses, vehicles, routes, tracking events and customer notifications. A layered object-oriented design prevents these concerns from being embedded in one tracking module and provides explicit extension points for new shipment types, route algorithms and delivery technologies.

Object-Oriented Principles

- Encapsulation keeps shipment state transitions, vehicle capacity and delivery status inside their owning objects/services, avoiding inconsistent updates from presentation code.
- Abstraction is achieved through RouteStrategy, DeliveryObserver and DAO interfaces. Business services depend on contracts rather than a particular algorithm, notification service or OODBMS API.
- Polymorphism supports StandardShipment, ExpressShipment, FragileShipment or future shipment classes through a common Shipment abstraction, while strategies supply interchangeable route-selection behavior.

Layered Architecture

- Presentation layer: ShipmentPortal and TrackingDashboard accept shipment requests and display status/ETA.
- Business layer: ShipmentService registers shipments; AllocationService assigns vehicles; TrackingService processes tracking events and coordinates notifications.
- Domain layer: Shipment, Warehouse, Vehicle, Delivery, Route and TrackingEvent hold logistics state and rules.
- Data access layer: ShipmentDAO, VehicleDAO and DeliveryDAO isolate OODBMS persistence and query operations.

Application of Design Patterns

- Strategy Pattern provides RoadRouteStrategy, FastestRouteStrategy or CostOptimizedRouteStrategy behind selectRoute(). Future AI-based optimization can implement the same strategy interface.

- Factory Pattern creates shipment subtypes with the correct handling rules. New types are introduced without changing every booking screen or service.
- Observer Pattern publishes delivery-status events to customer notifications, operations dashboards and exception-management components without hard-coded dependencies.
- DAO Pattern centralizes persistent operations for shipment, vehicle and delivery objects and avoids repeated OODBMS access code in business services.

Important Classes and Responsibilities

Class / Component	Key Attributes	Key Responsibilities / Methods
Shipment	shipmentId, type, weight, status	register(), changeStatus()
Vehicle	vehicleId, capacity, availability	canCarry(), allocate(), release()
Delivery	deliveryId, route, ETA, status	start(), updateStatus(), complete()
TrackingEvent	eventId, location, time, status	record()
AllocationService	vehicleDAO, routeStrategy	allocateVehicle(), planRoute()
ShipmentDAO	OODBMS access	findById(), save(), findByStatus()

OODBMS Interaction and Persistence Flow

When a shipment is registered, ShipmentService creates the correct Shipment through ShipmentFactory and persists it through ShipmentDAO. AllocationService retrieves available Vehicle objects, checks capacity, applies a RouteStrategy and creates a Delivery linking the Shipment and Vehicle.

The DAO layer stores these connected objects and subsequent TrackingEvent objects in the OODBMS. TrackingService retrieves the Delivery graph, applies a valid status transition, stores the update and then notifies observers. Because persistence is centralized, warehouse, vehicle and delivery queries can be optimized without changing the UI or route logic.

Evaluation of Quality Attributes

- Adaptability: route algorithms, shipment types and notification mechanisms are plug-in components.

- Scalability: warehouse and delivery services can be scaled independently, and new autonomous vehicle components can implement existing vehicle/allocation abstractions.
- Maintainability: object creation, routing, notification and persistence each have a clear owner, reducing duplicated code and conditional chains.
- Testability: route strategies and DAOs can be replaced with deterministic test implementations to validate allocation and tracking behavior.

UML / Layered Architecture Diagram

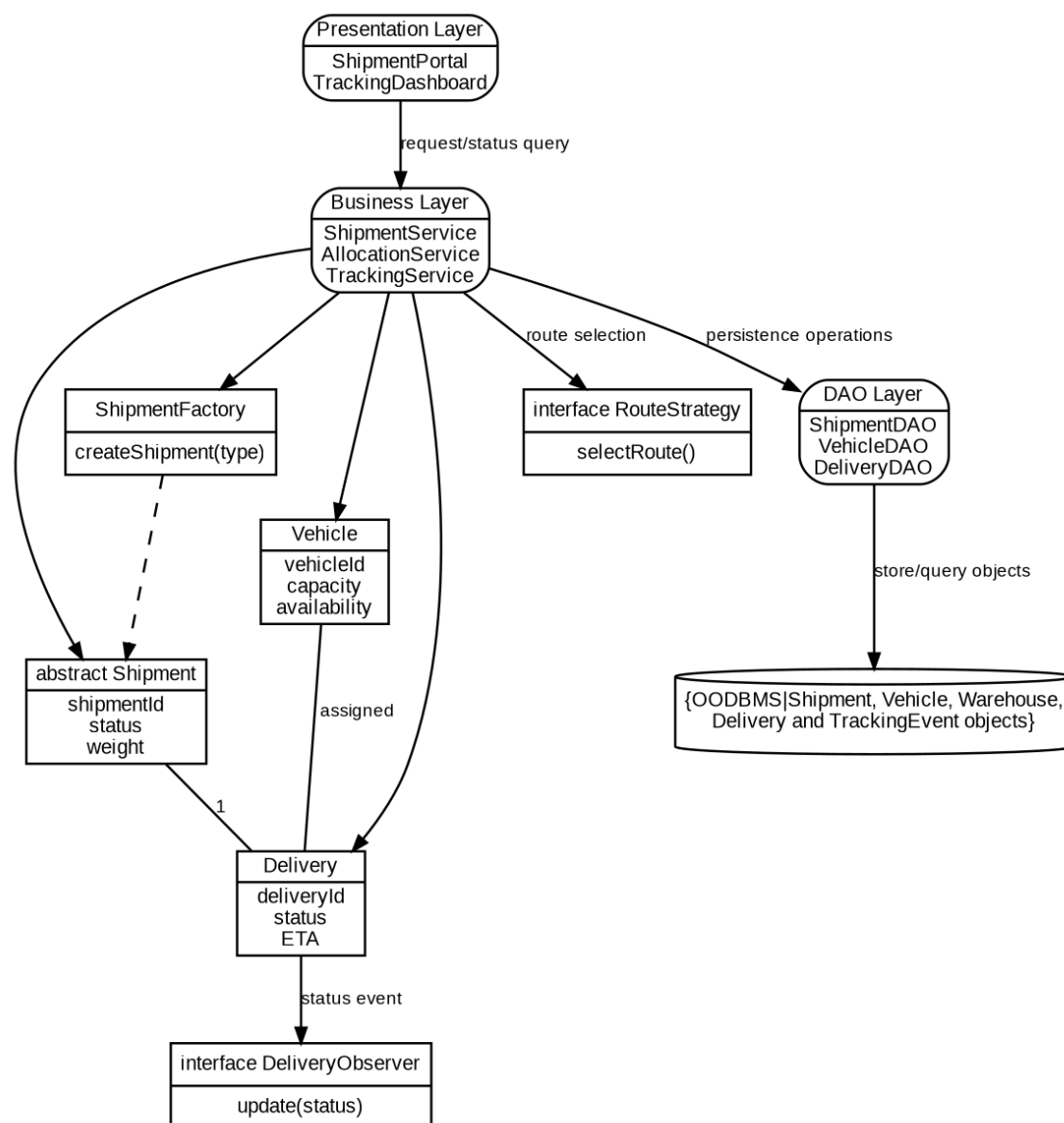


Figure 4: Object-oriented layered design for the system.

Conclusion

This architecture supports current delivery operations and also provides a direct migration path to autonomous vehicles and AI routing: they can be added as new implementations of existing abstractions instead of forcing structural changes across the application.

5. Smart Energy Management System

An energy management company develops a **Smart Energy Management System** to monitor electricity consumption, manage smart meters, analyze energy usage, and generate alerts for abnormal consumption. The system follows a layered object-oriented architecture where the presentation layer provides dashboards, the business layer performs energy analysis, the domain layer manages energy-related rules, and the data access layer communicates with an OODBMS.

The system uses the **Observer Pattern** for consumption alerts, the **Strategy Pattern** for different energy optimization algorithms, and the **Factory Pattern** for creating different smart-meter objects. A **Repository Pattern** is used to manage persistent energy data. The architecture is designed to accommodate future renewable-energy monitoring and AI-based consumption prediction.

Question:

Evaluate how object-oriented principles, layered architecture, and design patterns contribute to the scalability, extensibility, and maintainability of the energy management system. Explain the interaction between the system layers and the OODBMS for storing, retrieving, and managing persistent energy-monitoring objects.

Answer:

System Analysis

The energy system continuously collects smart-meter data, analyzes consumption and generates abnormal-usage alerts. Object-oriented layering is important because meter integration, analysis algorithms, domain data and persistence have different change cycles and performance requirements.

Object-Oriented Principles

- Encapsulation keeps meter configuration, reading values, consumption profiles and alert state behind controlled operations, reducing accidental modification of monitoring data.
- Abstraction through SmartMeter, OptimizationStrategy and repository interfaces prevents analysis services from depending on one meter vendor, one algorithm or one OODBMS API.
- Polymorphism allows different meter types and optimization algorithms to satisfy common interfaces. This follows the Open/Closed Principle: new renewable-energy meters or prediction strategies can be added without rewriting stable monitoring workflows.

Layered Architecture

- Presentation layer: EnergyDashboard and AdminConsole display live/aggregated usage, alerts and historical trends.
- Business layer: MonitoringService collects readings, AnalysisService creates consumption profiles and executes optimization strategies, and AlertService evaluates abnormal use.
- Domain layer: SmartMeter, EnergyReading, ConsumptionProfile, Alert and related rules represent the energy-monitoring model.
- Data access layer: MeterRepository, ReadingRepository and ProfileRepository isolate OODBMS storage, retrieval and object-lifecycle operations.

Application of Design Patterns

- Observer Pattern distributes abnormal-consumption events to customer alerts, operations teams and dashboards. Producers do not need direct references to every receiver.
- Strategy Pattern encapsulates different optimization algorithms such as peak-load reduction, tariff-aware scheduling or later AI-based prediction. The business service selects an implementation through the common interface.
- Factory Pattern creates different SmartMeter implementations according to meter type/vendor/protocol and prevents hardware-specific construction logic from spreading through the application.
- Repository Pattern provides stable operations for storing and querying readings, profiles and meter objects while keeping OODBMS APIs out of business code.

Important Classes and Responsibilities

Class / Component	Key Attributes	Key Responsibilities / Methods
SmartMeter	meterId, type, location	readConsumption(), getStatus()
EnergyReading	timestamp, kWh, voltage	validate(), normalize()
ConsumptionProfile	customerId, period, averageUsage	calculateTrend(), compareBaseline()
Alert	alertId, severity, message	raise(), acknowledge()
AnalysisService	strategy, repositories	analyzeUsage(), optimizeConsumption()
ReadingRepository	OODBMS access	save(), findByMeter(), findByPeriod()

OODBMS Interaction and Persistence Flow

MonitoringService receives data through the SmartMeter abstraction and creates EnergyReading objects. After validation, ReadingRepository persists them. AnalysisService retrieves readings for a meter/time period, builds or updates a ConsumptionProfile and applies the selected OptimizationStrategy. If usage violates a rule, AlertService publishes an event to registered observers.

Repository implementations manage OODBMS sessions, object identities, relationships and transaction boundaries. A dashboard request therefore follows Presentation -> Business -> Repository -> OODBMS and returns persistent meter/profile objects through the same path. High-volume reading queries remain confined to repositories, where indexing, caching or batching can be optimized without changing analysis logic.

Evaluation of Quality Attributes

- Scalability: collection, analysis and presentation responsibilities are separable, and repositories can be optimized for increasing meter/read volumes.
- Extensibility: renewable-energy meters, new optimization policies and AI prediction engines can implement existing interfaces.
- Maintainability: device-specific, algorithmic and persistence changes remain localized to their corresponding components.
- Reliability/testability: services can be tested with fake meters and in-memory repositories, while observers allow alerts to evolve independently of monitoring logic.

UML / Layered Architecture Diagram

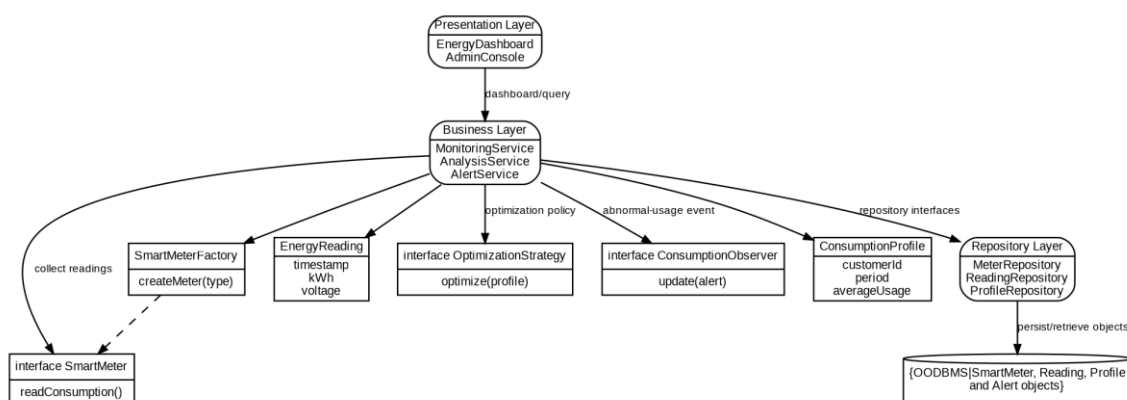


Figure 5: Object-oriented layered design for the system.

Conclusion

The resulting design gives the energy platform a stable object model and clean OODBMS access path while keeping meter technologies and optimization algorithms replaceable. It is therefore suitable for future renewable-energy monitoring and AI-based consumption prediction.

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation